AI-BASED HANDWRITTEN DIGIT RECOGNITION SYSTEM

Major Project Report

Submitted by: [Your Full Name]
Programme: B.Tech – Computer Science and Engineering
Institution: National Institute of Technology Nagaland
Academic Year: [Enter Academic Year]

Abstract

This project presents an AI-based Handwritten Digit Recognition System that classifies handwritten digits from 0 to 9. A Convolutional Neural Network (CNN) is trained using the MNIST dataset. A Flask web application allows users to upload an image or draw a digit on a browser canvas. The input is preprocessed into a 28 × 28 grayscale image and passed to the trained model. The application displays the predicted digit and confidence score.

1. Introduction

Handwritten digit recognition is a computer vision task in which a computer identifies numerical characters written by a person. Handwriting varies in shape, size, thickness, and style. This project uses deep learning, specifically a CNN, to learn visual patterns from labelled digit images.

2. Problem Statement

The objective is to develop a system that recognizes a single handwritten digit (0–9) from an image, preprocesses the image, classifies it using a trained model, and displays the result through a web interface.

3. Objectives

Load and explore MNIST; preprocess images for CNN input; build, train, and evaluate a CNN; visualize performance using accuracy/loss plots and a confusion matrix; develop a Flask interface for image upload and canvas drawing; display the predicted digit and confidence score.

4. Dataset Description

MNIST contains 70,000 grayscale handwritten digit images: 60,000 training images and 10,000 test images. Each image is 28 × 28 pixels and belongs to one of ten classes (0–9). Pixel values range from 0 to 255 before normalization.

5. Data Preprocessing

MNIST pixel values are converted to floating point and divided by 255 to obtain values in the range 0–1. A channel dimension is added, giving the shape (number_of_images, 28, 28, 1). For user images, preprocessing includes reading, grayscale conversion, contrast improvement, thresholding, noise removal, cropping, padding, aspect-ratio-preserving resizing, placement on a 28 × 28 canvas, normalization, and reshaping to (1, 28, 28, 1).

6. CNN Model Architecture

Input 28 × 28 × 1; Conv2D with 32 filters and a 3 × 3 kernel (ReLU); MaxPooling2D 2 × 2; Conv2D with 64 filters and a 3 × 3 kernel (ReLU); MaxPooling2D 2 × 2; Flatten; Dense with 128 units (ReLU); Dropout 0.3; Dense output with 10 units (Softmax). The Softmax layer produces class probabilities for digits 0–9.

7. Model Training

The model is compiled with the Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric. The reviewed training notebook used batch size 64, five epochs, and a 10% validation split. Verify these settings against the final notebook before submission.

8. Evaluation and Results

The previously reviewed notebook run showed approximately 98.97% test accuracy and approximately 0.0332 test loss. Replace these figures if the final run differs. Add the final accuracy/loss graphs and confusion matrix from the notebook. MNIST test performance may differ from performance on photographs because external images vary in background, lighting, stroke thickness, positioning, and writing style.

9. Web Application Design

The frontend uses HTML, CSS, and JavaScript, while Flask handles backend requests. The home route displays the interface. The /health route reports service status. The /predict route validates an uploaded file, saves it temporarily, preprocesses it, runs model prediction, and returns the predicted digit and confidence as JSON. The temporary image is deleted after processing. The browser interface supports upload and drawing.

10. How to Run and Use

From the project directory, create and activate a Python virtual environment, install dependencies with `pip install -r requirements.txt`, and run `python app.py`. Open `http://127.0.0.1:5000`. Upload a PNG/JPG/JPEG image containing one digit and click the upload prediction button, or draw a digit on the canvas and click the drawing prediction button. Use Clear to reset the canvas.

11. Tools and Technologies

Python, TensorFlow/Keras, NumPy, OpenCV, Flask, HTML, CSS, JavaScript, Jupyter Notebook, and the MNIST dataset.

12. Limitations and Future Scope

The current system recognizes one digit and is trained on MNIST. Complex backgrounds, uneven lighting, unusual handwriting, and multiple digits may reduce accuracy. Future work may include collecting a broader real-world dataset, testing data augmentation, improving segmentation, adding multi-digit recognition, and deploying the application.

13. Conclusion

The project demonstrates an end-to-end handwritten digit recognition workflow, from dataset exploration and preprocessing to CNN training, evaluation, and web application integration. The Flask interface makes the trained model accessible through image upload and canvas drawing.

References

1. MNIST handwritten digit database, distributed through TensorFlow/Keras datasets. 2. TensorFlow/Keras documentation: tensorflow.org and keras.io. 3. Flask documentation: flask.palletsprojects.com. 4. OpenCV documentation: docs.opencv.org.

Before submission: Fill in your name and academic year, verify the final accuracy/loss from your latest notebook run, and insert screenshots of the working application, training graphs, and confusion matrix.